LAB ACTIVITY 2: Hub-and-Spoke API Integration
Understanding Hub-and-Spoke Integration Using Postman/Talend API Tester

ACTIVITY OVERVIEW
What You Will Do
In this activity, you will:

1. Build 3 Mock Services (Spokes) - Simple APIs that return data
2. Build 1 API Gateway (Hub) - Aggregates data from all services
3. Test Everything Using Postman - Verify each endpoint works
4. Draw the Architecture - Visualize the Hub-and-Spoke pattern

Why This Approach?
Focus on BACKEND logic only - no HTML distraction
Verify each component works INDEPENDENTLY
Test the INTEGRATION before building the UI
Easier to DEBUG - see exactly what's happening
Build CONFIDENCE - see it working before adding complexity
Professional workflow - this is how real developers work!

LEARNING OBJECTIVES

	Objective
1	Build 3 mock services (spokes) using Node.js
2	Build 1 API gateway (hub) that aggregates data
3	Test each service independently using Postman
4	Test the aggregated dashboard endpoint
5	Understand the Hub-and-Spoke integration pattern
6	Draw and label the system architecture

Why This Pattern?

Benefit	Explanation
Centralized Control	All requests go through one place
Simplified Management	Only need to update one gateway
Loose Coupling	Spokes can change independently
Security	One place to enforce security
Monitoring	One place to log all requests

WHAT YOU WILL BUILD

The Services (Spokes)

Service	Port	Endpoint	What It Returns
User Service	5001	/users	List of users with id, name, email
Product Service	5002	/products	List of products with id, name, price
Order Service	5003	/orders	List of orders with id, userId, status

The Gateway (Hub)

Endpoint	Method	Description
/api/dashboard	GET	Returns data from ALL 3 services combined
/api/status	GET	Checks if all services are running
/api/users	GET	Returns only user data
/api/products	GET	Returns only product data
/api/orders	GET	Returns only order data

PROJECT STRUCTURE

```
Lab2-hubspoke/
│
├── services/
│   ├── user-service/
│   │   ├── server.js
│   │   └── package.json
│   ├── product-service/
│   │   ├── server.js
│   │   └── package.json
│   └── order-service/
│       ├── server.js
│       └── package.json
│
├── gateway/
    ├── server.js
    └── package.json
```

STEP-BY-STEP INSTRUCTIONS
Step 1: Create the Project Folder using this commands on CMD, or use VSCODE.

```
# Create the main folder
mkdir lab-2a-hub-spoke
cd lab-2a-hub-spoke

# Create subfolders
mkdir -p services/user-service
mkdir -p services/product-service
mkdir -p services/order-service
mkdir gateway
```

Step 2: Create the User Service (Spoke 1)
File: services/user-service/server.js

```
const express = require('express');
const app = express();
const PORT = 5001;

// Sample data - 5 users
const users = [
    { id: 1, name: 'Alice Johnson', email: 'alice@email.com',
role: 'Admin' },
    { id: 2, name: 'Bob Smith', email: 'bob@email.com', role:
'User' },
    { id: 3, name: 'Carol Davis', email: 'carol@email.com',
role: 'Editor' },
    { id: 4, name: 'David Wilson', email: 'david@email.com',
role: 'User' },
    { id: 5, name: 'Eva Martinez', email: 'eva@email.com',
role: 'User' }
];

// Endpoint: GET /users
app.get('/users', (req, res) => {
    console.log('User Service: Returning', users.length,
'users');
    res.json({
        service: 'User Service',
        status: 'success',
        count: users.length,
        data: users,
        timestamp: new Date().toISOString()
    });
});

// Health check: GET /ping
app.get('/ping', (req, res) => {
    res.json({ status: 'online', service: 'User Service' });
});

// Start server
app.listen(PORT, () => {
    console.log('=' .repeat(40));
    console.log('USER SERVICE STARTED');
    console.log('=' .repeat(40));
```

```
    console.log(`Port: ${PORT}`);
    console.log(`GET /users - Returns ${users.length} users`);
    console.log(`GET /ping - Health check`);
    console.log('=' .repeat(40));
});
cd
```

File: services/user-service/package.json

```
{
    "name": "user-service",
    "version": "1.0.0",
    "description": "Mock User Service - Spoke 1",
    "main": "server.js",
    "scripts": {
        "start": "node server.js"
    },
    "dependencies": {
        "express": "^4.18.2"
    }
}
```

Step 3: Create the Product Service (Spoke 2)

File: services/product-service/server.js

```
const express = require('express');
const app = express();
const PORT = 5002;

// Sample data - 5 products
const products = [
    { id: 1, name: 'Laptop Pro', price: 1299.99, category:
'Electronics', inStock: true },
    { id: 2, name: 'Wireless Mouse', price: 29.99, category:
'Accessories', inStock: true },
    { id: 3, name: 'USB-C Cable', price: 19.99, category:
'Accessories', inStock: false },
    { id: 4, name: 'Monitor 27"', price: 349.99, category:
'Electronics', inStock: true },
    { id: 5, name: 'Keyboard Pro', price: 89.99, category:
'Accessories', inStock: true }
];

// Endpoint: GET /products
app.get('/products', (req, res) => {
```

```
    console.log('Product Service: Returning', products.length,
'products');
    res.json({
        service: 'Product Service',
        status: 'success',
        count: products.length,
        data: products,
        timestamp: new Date().toISOString()
    });
});

// Health check: GET /ping
app.get('/ping', (req, res) => {
    res.json({ status: 'online', service: 'Product Service'
});
});

// Start server
app.listen(PORT, () => {
    console.log('=' .repeat(40));
    console.log('PRODUCT SERVICE STARTED');
    console.log('=' .repeat(40));
    console.log(`Port: ${PORT}`);
    console.log(`GET /products - Returns ${products.length}
products`);
    console.log(`GET /ping - Health check`);
    console.log('=' .repeat(40));
});
```

File: services/product-service/package.json

```
json
{
    "name": "product-service",
    "version": "1.0.0",
    "description": "Mock Product Service - Spoke 2",
    "main": "server.js",
    "scripts": {
        "start": "node server.js"
    },
    "dependencies": {
        "express": "^4.18.2"
    }
}
```

Step 4: Create the Order Service (Spoke 3)
File: services/order-service/server.js

```
const express = require('express');
const app = express();
const PORT = 5003;

// Sample data - 5 orders
const orders = [
    { id: 1, userId: 1, productId: 1, status: 'Delivered',
total: 1299.99 },
    { id: 2, userId: 2, productId: 3, status: 'Shipped',
total: 19.99 },
    { id: 3, userId: 3, productId: 2, status: 'Processing',
total: 29.99 },
    { id: 4, userId: 1, productId: 4, status: 'Delivered',
total: 349.99 },
    { id: 5, userId: 5, productId: 5, status: 'Pending',
total: 89.99 }
];

// Endpoint: GET /orders
app.get('/orders', (req, res) => {
    console.log('Order Service: Returning', orders.length,
'orders');
    res.json({
        service: 'Order Service',
        status: 'success',
        count: orders.length,
        data: orders,
        timestamp: new Date().toISOString()
    });
});

// Health check: GET /ping
app.get('/ping', (req, res) => {
    res.json({ status: 'online', service: 'Order Service' });
});

// Start server
app.listen(PORT, () => {
    console.log('=' .repeat(40));
    console.log('ORDER SERVICE STARTED');
    console.log('=' .repeat(40));
```

```
    console.log(`Port: ${PORT}`);
    console.log(`GET /orders - Returns ${orders.length}
orders`);
    console.log(`GET /ping - Health check`);
    console.log('=' .repeat(40));
});
```

File: services/order-service/package.json

```
json
{
    "name": "order-service",
    "version": "1.0.0",
    "description": "Mock Order Service - Spoke 3",
    "main": "server.js",
    "scripts": {
        "start": "node server.js"
    },
    "dependencies": {
        "express": "^4.18.2"
    }
}
```

Step 5: Create the API Gateway (Hub)

File: gateway/server.js

```
const express = require('express');
const axios = require('axios');
const cors = require('cors');

const app = express();
const PORT = 3000;

//
==========================================================
==
// SERVICE URLs (The Spokes)
//
==========================================================
==

const SERVICES = {
    users: 'http://localhost:5001/users',
    products: 'http://localhost:5002/products',
    orders: 'http://localhost:5003/orders'
};
```

```
//
================================================================
==
// MIDDLEWARE
//
================================================================
==

app.use(cors());        // Allow cross-origin requests
app.use(express.json()); // Parse JSON bodies

//
================================================================
==
// ROUTE 1: Dashboard - Aggregates ALL Services
//
================================================================
==

app.get('/api/dashboard', async (req, res) => {
    console.log('Dashboard requested - aggregating data...');

    try {
        // Call ALL services in PARALLEL (Promise.all)
        const [usersRes, productsRes, ordersRes] = await
Promise.all([
            axios.get(SERVICES.users),
            axios.get(SERVICES.products),
            axios.get(SERVICES.orders)
        ]);

        // Combine all data into one response
        const dashboard = {
            timestamp: new Date().toISOString(),
            summary: {
                totalUsers: usersRes.data.count || 0,
                totalProducts: productsRes.data.count || 0,
                totalOrders: ordersRes.data.count || 0
            },
            users: usersRes.data.data || [],
            products: productsRes.data.data || [],
            orders: ordersRes.data.data || []
        };
```

```
        console.log(`Dashboard:
${dashboard.summary.totalUsers} users,
${dashboard.summary.totalProducts} products,
${dashboard.summary.totalOrders} orders`);
        res.json(dashboard);

    } catch (error) {
        console.error('Aggregation error:', error.message);
        res.status(500).json({
            error: 'Failed to fetch data from one or more
services',
            details: error.message
        });
    }
});

//
================================================================
==
// ROUTE 2: Check Service Status
//
================================================================
==

app.get('/api/status', async (req, res) => {
    console.log('Checking service status...');

    try {
        const results = await Promise.all([
            axios.get('http://localhost:5001/ping').then(() =>
({ name: 'users', status: 'online' })).catch(() => ({ name:
'users', status: 'offline' })),
            axios.get('http://localhost:5002/ping').then(() =>
({ name: 'products', status: 'online' })).catch(() => ({ name:
'products', status: 'offline' })),
            axios.get('http://localhost:5003/ping').then(() =>
({ name: 'orders', status: 'online' })).catch(() => ({ name:
'orders', status: 'offline' }))
        ]);

        const allOnline = results.every(s => s.status ===
'online');
        res.json({
```

```
            gateway: 'online',
            allOnline: allOnline,
            services: results
        });

    } catch (error) {
        res.status(500).json({ error: 'Status check failed'
});
    }
});

//
==========================================================
==
// ROUTE 3: Individual Service Endpoints
//
==========================================================
==

app.get('/api/users', async (req, res) => {
    try {
        const response = await axios.get(SERVICES.users);
        res.json(response.data);
    } catch (error) {
        res.status(500).json({ error: 'Failed to fetch users'
});
    }
});

app.get('/api/products', async (req, res) => {
    try {
        const response = await axios.get(SERVICES.products);
        res.json(response.data);
    } catch (error) {
        res.status(500).json({ error: 'Failed to fetch
products' });
    }
});

app.get('/api/orders', async (req, res) => {
    try {
        const response = await axios.get(SERVICES.orders);
        res.json(response.data);
    } catch (error) {
```

```
        res.status(500).json({ error: 'Failed to fetch orders'
});
    }
});

//
==========================================================
==
// START THE GATEWAY
//
==========================================================
==

app.listen(PORT, () => {
    console.log('=' .repeat(50));
    console.log('API GATEWAY (HUB) STARTED');
    console.log('=' .repeat(50));
    console.log(`Gateway URL: http://localhost:${PORT}`);
    console.log('=' .repeat(50));
    console.log('ENDPOINTS:');
    console.log(`Dashboard:
http://localhost:${PORT}/api/dashboard`);
    console.log(`Status:
http://localhost:${PORT}/api/status`);
    console.log(`Users:
http://localhost:${PORT}/api/users`);
    console.log(`Products:
http://localhost:${PORT}/api/products`);
    console.log(`Orders:
http://localhost:${PORT}/api/orders`);
    console.log('=' .repeat(50));
    console.log('All services will be called in PARALLEL!');
    console.log('Open Postman and test the endpoints.');
});
```

File: gateway/package.json
json
```
{
    "name": "api-gateway",
    "version": "1.0.0",
    "description": "API Gateway (Hub) that aggregates multiple
services",
    "main": "server.js",
    "scripts": {
        "start": "node server.js"
    },
    "dependencies": {
        "axios": "^1.6.0",
        "cors": "^2.8.5",
        "express": "^4.18.2"
    }
}
```

RUNNING THE SYSTEM

Step 1: Install Dependencies

```
# Install dependencies for each service

# User Service
cd services/user-service
npm install

# Product Service
cd ../product-service
npm install

# Order Service
cd ../order-service
npm install

# Gateway
cd ../../gateway
npm install
```

Step 2: Start All Services

Open 4 separate terminals:

bash
```
# Terminal 1 - User Service (Spoke 1)
cd services/user-service
```

```
node server.js

# Terminal 2 - Product Service (Spoke 2)
cd services/product-service
node server.js

# Terminal 3 - Order Service (Spoke 3)
cd services/order-service
node server.js

# Terminal 4 - API Gateway (Hub)
cd gateway
node server.js
```

Step 3: Expected Terminal Outputs

Terminal 1 - User Service:

```
==========================================
USER SERVICE STARTED
==========================================
Port: 5001
GET /users - Returns 5 users
GET /ping - Health check
==========================================
```

Terminal 2 - Product Service:

```
==========================================
PRODUCT SERVICE STARTED
==========================================
Port: 5002
GET /products - Returns 5 products
GET /ping - Health check
==========================================
```

Terminal 3 - Order Service:

```
==========================================
ORDER SERVICE STARTED
==========================================
Port: 5003
GET /orders - Returns 5 orders
GET /ping - Health check
==========================================
```

Terminal 4 - API Gateway:

```
================================================
```

```
API GATEWAY (HUB) STARTED
==================================================
URL: http://localhost:3000
==================================================
ENDPOINTS:
   Dashboard: http://localhost:3000/api/dashboard
   Status:    http://localhost:3000/api/status
   Users:     http://localhost:3000/api/users
   Products:  http://localhost:3000/api/products
   Orders:    http://localhost:3000/api/orders
==================================================
All services will be called in PARALLEL!
```

Open Postman and test the endpoints.

POSTMAN TESTING

Step 1: Download Postman

If you don't have Postman installed:

- Go to: https://www.postman.com/downloads/
- Download and install the desktop app

Step 2: Test Each Service Directly

Test 1: User Service

Field	Value
Method	GET
URL	http://localhost:5001/users

Expected Response:


```
{
    "service": "User Service",
    "status": "success",
    "count": 5,
    "data": [
        { "id": 1, "name": "Alice Johnson", "email":
"alice@email.com", "role": "Admin" },
        { "id": 2, "name": "Bob Smith", "email":
"bob@email.com", "role": "User" },
        { "id": 3, "name": "Carol Davis", "email":
"carol@email.com", "role": "Editor" },
        { "id": 4, "name": "David Wilson", "email":
"david@email.com", "role": "User" },
```

```
        { "id": 5, "name": "Eva Martinez", "email":
"eva@email.com", "role": "User" }
    ],
    "timestamp": "2024-01-15T14:30:00.000Z"
}
```

Test 2: Product Service

Field	Value
Method	GET
URL	http://localhost:5002/products

Expected Response:


```
{
    "service": "Product Service",
    "status": "success",
    "count": 5,
    "data": [
        { "id": 1, "name": "Laptop Pro", "price": 1299.99,
"category": "Electronics", "inStock": true },
        { "id": 2, "name": "Wireless Mouse", "price": 29.99,
"category": "Accessories", "inStock": true },
        { "id": 3, "name": "USB-C Cable", "price": 19.99,
"category": "Accessories", "inStock": false },
        { "id": 4, "name": "Monitor 27\"", "price": 349.99,
"category": "Electronics", "inStock": true },
        { "id": 5, "name": "Keyboard Pro", "price": 89.99,
"category": "Accessories", "inStock": true }
    ],
    "timestamp": "2024-01-15T14:30:00.000Z"
}
```


Test 3: Order Service

Field	Value
Method	GET
URL	http://localhost:5003/orders

Expected Response:


```
{
    "service": "Order Service",
    "status": "success",
    "count": 5,
    "data": [
        { "id": 1, "userId": 1, "productId": 1, "status":
"Delivered", "total": 1299.99 },
```

```
        { "id": 2, "userId": 2, "productId": 3, "status":
"Shipped", "total": 19.99 },
        { "id": 3, "userId": 3, "productId": 2, "status":
"Processing", "total": 29.99 },
        { "id": 4, "userId": 1, "productId": 4, "status":
"Delivered", "total": 349.99 },
        { "id": 5, "userId": 5, "productId": 5, "status":
"Pending", "total": 89.99 }
    ],
    "timestamp": "2024-01-15T14:30:00.000Z"
}
```

Step 3: Test the Gateway (Hub)
Test 4: Dashboard (Aggregated Data)

Field	Value
Method	GET
URL	http://localhost:3000/api/dashboard

Expected Response:

```
{
    "timestamp": "2024-01-15T14:30:00.000Z",
    "summary": {
        "totalUsers": 5,
        "totalProducts": 5,
        "totalOrders": 5
    },
    "users": [
        { "id": 1, "name": "Alice Johnson", "email":
"alice@email.com", "role": "Admin" },
        { "id": 2, "name": "Bob Smith", "email":
"bob@email.com", "role": "User" },
        ...
    ],
    "products": [
        { "id": 1, "name": "Laptop Pro", "price": 1299.99,
"category": "Electronics", "inStock": true },
        ...
    ],
    "orders": [
        { "id": 1, "userId": 1, "productId": 1, "status":
"Delivered", "total": 1299.99 },
```

```
        ...
    ]
}
```

Test 5: Status Check

Field	Value
Method	GET
URL	http://localhost:3000/api/status

Expected Response:

```
json
{
    "gateway": "online",
    "allOnline": true,
    "services": [
        { "name": "users", "status": "online" },
        { "name": "products", "status": "online" },
        { "name": "orders", "status": "online" }
    ]
}
```

Test 6: Individual Endpoints via Gateway

Endpoint	URL
Users	http://localhost:3000/api/users
Products	http://localhost:3000/api/products
Orders	http://localhost:3000/api/orders

LAB ACTIVITY 2: STUDENT ACTIVITY SHEET

Name: Naza,Angelica M.
Course/Year: BSIT-3E
Date: SEPTEMBER,26 2026

What is Hub-and-Spoke Integration?
Hub-and-Spoke integration is a way of connecting different services using one main system called the hub. The other services are called spokes,they communicate though the hub.

Part 1: Postman Testing Results (30 points)

Test	URL	Status Code	Did it work?
1. User Service	http://localhost:5001/users	200	☑ Yes ☐ No
2. Product Service	http://localhost:5002/products	200	☑ Yes ☐ No
3. Order Service	http://localhost:5003/orders	200	☑ Yes ☐ No
4. Dashboard	http://localhost:3000/api/dashboard	200	☑ Yes ☐ No
5. Status	http://localhost:3000/api/status	200	☑ Yes ☐ No

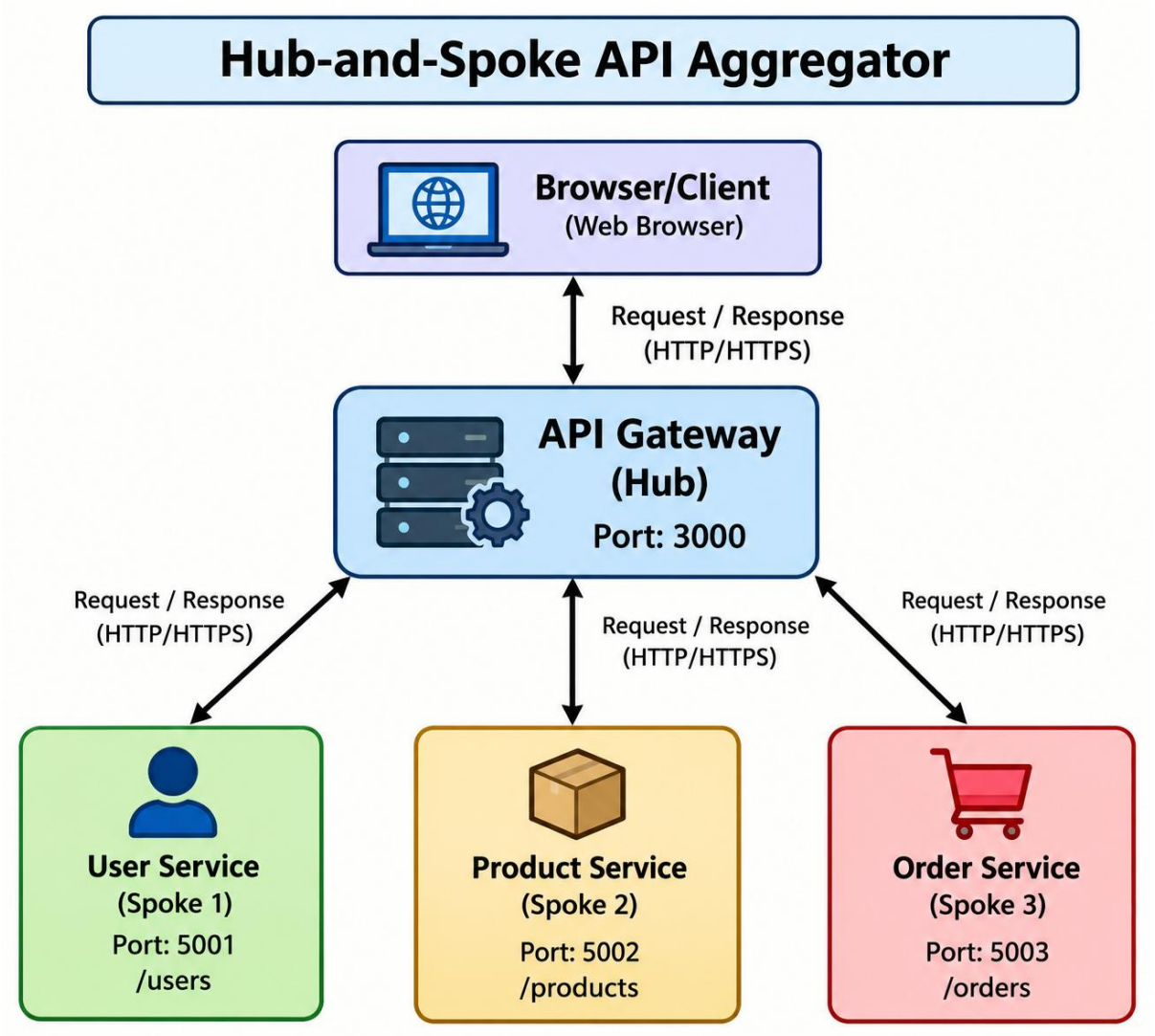
Part 2: Draw the Architecture (40 points)
DRAWING THE ARCHITECTURE

Now, Draw the Architecture You Just Built!
After testing everything with Postman, draw the architecture diagram on the space below.

Your drawing MUST include:

Element	Description
1	Browser/Client (with label)
2	API Gateway (Hub) - Port 3000
3	User Service (Spoke 1) - Port 5001, /users
4	Product Service (Spoke 2) - Port 5002, /products
5	Order Service (Spoke 3) - Port 5003, /orders
6	Arrows showing data flow direction
7	All labels clearly written
8	Title: "Hub-and-Spoke API Aggregator"

Draw your architecture inside the box



Part 3: Label Your Diagram (20 points)

Component	What does it do?	Port
API Gateway (Hub)	Connect and get data from the differentservices	3000
User Service (Spoke 1)	Provide user information	5001
Product Service (Spoke 2)	Provide product information	5002
Order Service (Spoke 3)	Provide order information	5003

Part 4: Integration Pattern Questions (10 points)

1. What integration pattern did you use?

Hub-and-spoke integration

2. What is the "Hub" in this architecture?

The API gateway is in the hub

3. What are the "Spokes" in this architecture?

The user service, product service, and order service and the spokes.

4. Why is this better than Point-to-Point integration?

It is the easier to manage because all the services connect through one hub instead of connecting to each other directly.

QUICK REFERENCE: COMMANDS

Task	Command
Install dependencies (user)	cd services/user-service && npm install
Install dependencies (product)	cd services/product-service && npm install
Install dependencies (order)	cd services/order-service && npm install
Install dependencies (gateway)	cd gateway && npm install
Start User Service	cd services/user-service && node server.js
Start Product Service	cd services/product-service && node server.js
Start Order Service	cd services/order-service && node server.js
Start Gateway	cd gateway && node server.js

COMPLETION CHECKLIST

Before submitting, make sure you have:

- ☐ Created all 3 service folders
- ☐ Created the gateway folder
- ☐ Installed all dependencies (npm install)
- ☐ Started all 4 servers (4 terminals)
- ☐ Tested User Service in Postman
- ☐ Tested Product Service in Postman
- ☐ Tested Order Service in Postman
- ☐ Tested Dashboard endpoint in Postman
- ☐ Tested Status endpoint in Postman
- ☐ Drawn the architecture diagram
- ☐ Labeled all components
- ☐ Answered all questions

CONGRATULATIONS!

You've Successfully Built a Hub-and-Spoke Integration!

WHAT YOU ACCOMPLISHED:

☑ Built 3 independent services (spokes)
☑ Built 1 API gateway (hub)
☑ Tested everything with Postman
☑ Drawn the architecture
☑ UNDERSTOOD Hub-and-Spoke Integration!

This is how real-world systems work!

Next: We'll add the HTML dashboard to visualize the data!